Real Time Fraud Detection Model

Detailed Documentation of Model Building Process

Prepared by: Xueshan (Kevin) Peng

11/5/2025

Table of Contents

1. **Executive Summary**
2. **Description of the Data**
    - Data Overview
    - Summary Statistics
3. **Data Cleaning**
    - Data Exclusions
    - Outlier Treatment
    - Imputation Methods
4. **Variable Creation**
5. **Feature Selection**
    - Methodology
    - Final Variable List
6. **Preliminary Model Explores**
    - Algorithm Descriptions
    - Comparative Performance Table
7. **Final Model Performance**
    - Final Model Architecture and Hyperparameters
    - Results: Training Set, Testing Set, Out-of-Time (OOT) Set
8. **Financial Curves and Recommended Cutoff**
    - Financial Impact Plots
    - Recommended Cutoff Strategy
9. **Summary**
10. **Appendix**
    - Full Data Quality Report (DQR)

Executive Summary

A predictive fraud model was developed to proactively identify and stop fraudulent card transactions. This involved analyzing historical transaction data to build behavioral and velocity-based features that can detect spending anomalies. The final model demonstrates strong, stable performance on future, out-of-time (OOT) data.

The model's business value is significant. It is highly efficient, achieving a **Fraud Detection Rate (FDR) of 62.64% while reviewing only the top 3%** of the highest-risk transactions. Based on the full financial analysis, the recommended implementation strategy, flagging the top 6% of transactions, is projected to generate **$60.2 million in net savings**. This approach optimally balances maximizing fraud capture with managing operational costs and ensuring a positive customer experience.

Description of the Data

Data Overview

The dataset is **Card Transactions Data**, which includes merchant identifiers, transaction types, and a binary fraud label. The data came from around 100,000 real U.S. applications **over 8 months**. There are **10 fields** and **98,393 records**.

Summary Statistics

Below are a summarization of all the variables in the original dataset.

Fig 1a. Numeric Fields Table

Field Name	Field Type	# Records Have Values	% Populated	# Zeros	Min	Max	Mean	Standard Deviation	Most Common
Amount	numeric	98393	100.00%	0	0.01	3102046	424.2909	9922.44	3.62

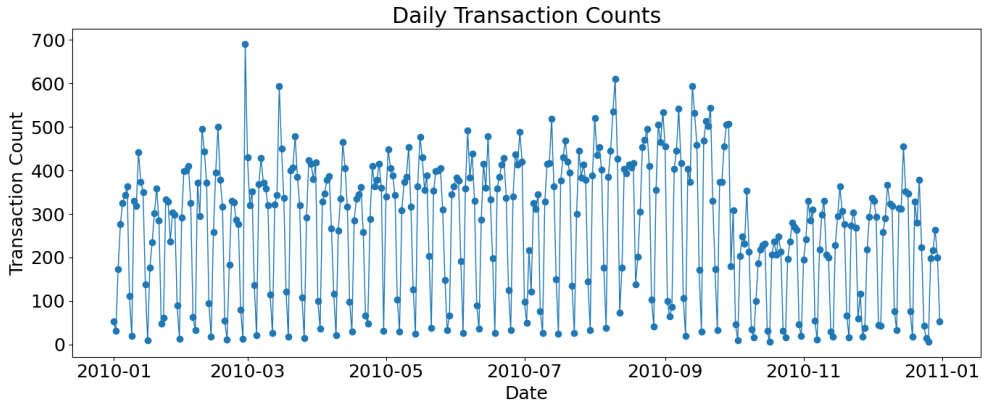
Fig 1b. Categorical Fields Table

Field Name	Field Type	# Records Have Values	% Populated	# Zeros	# Unique Values	Most Common
Date	Categorical	98393	100.00%	0	365	2/28/10
Merchnum	Categorical	94970	96.52%	0	13091	930090121224
Merch description	Categorical	98393	100.00%	0	13126	GSA-FSS-ADV
Merch state	Categorical	97181	98.77%	0	227	TN
Transtype	Categorical	98393	100.00%	0	4	P
Recnum	Categorical	98393	100.00%	0	98393	1
Fraud	Categorical	98393	100.00%	95901	2	0
Merch zip	Categorical	93664	95.19%	0	4567	38118
Cardnum	Categorical	98393	100.00%	0	1645	5142148452

Key Insights

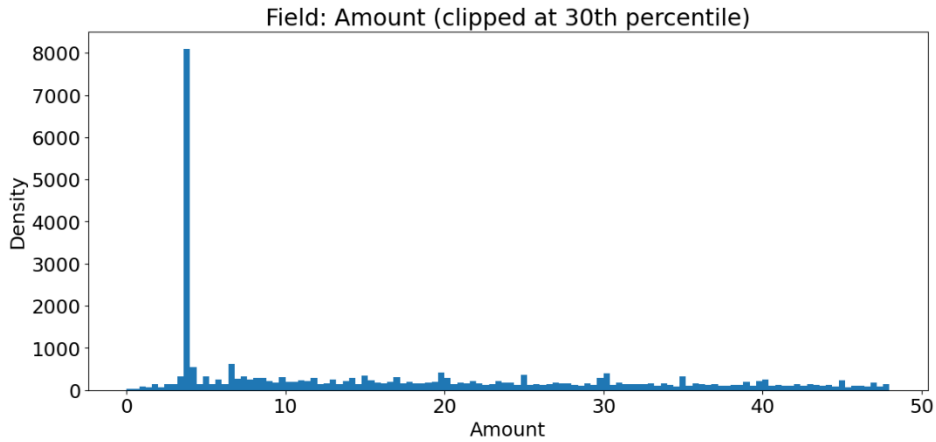
Transaction volume exhibits a strong weekly pattern. A significant decline in transactions occurs around October, which aligns with the renewal of the fiscal year. This suggests that business spending increases before the renewal and becomes more conservative immediately after.

Fig 2a. Daily Transaction Counts



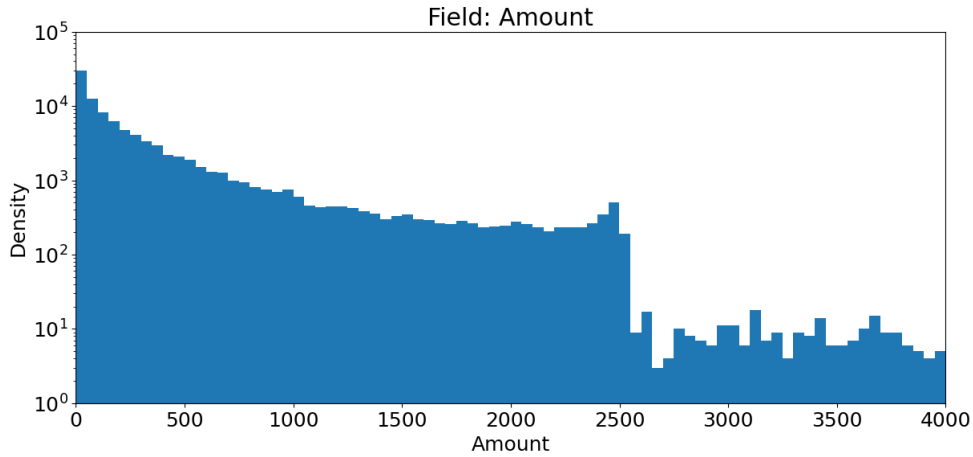
A notable anomaly was detected in the transaction data: a high-frequency spike at the **$3.62** price point. Investigation confirms these transactions primarily consist of uniform charges from **FedEx**.

Fig 2b. Transaction Amount Distribution



Transaction amounts show a sharp decline at the $2,500 mark. This pattern is consistent with most companies having an internal policy requiring manager approval for expenditures above this threshold.

Fig 2c. Transaction Amount Distribution



Data Cleaning

Data Exclusions

The data cleaning process began by addressing transactions that required exclusion from the main dataset. Specifically, any transaction that had a Merchnum value of "0" was converted to a missing value. This step ensures these records do not skew analysis and are correctly handled during imputation.

Outlier Treatment

A rule-based outlier flag, **amount_okay**, was created to identify and isolate a known set of anomalous transactions. Transactions were flagged as anomalous (False) if their **Merch description** began with "FEDEX" and the transaction **Amount** was between $3.62 and $3.80, inclusive. All other transactions were flagged as True (okay).

Imputation Methods

A multi-step process was used to standardize merchant information and impute missing location data. First, merchant identification numbers were standardized: transactions with a **Merch description** indicating an "adjustment" had their **Merchnum** set to "Unknown". Additionally, new canonical (standardized) **Merchnum** values were generated for any previously unseen **Merch descriptions** to ensure consistency.

For location imputation, a hierarchical approach was used. **Merch state** was first populated using a **Merch zip**-to-state lookup. If the state remained missing, it was imputed using the first observed state associated with the same **Merchnum** or, failing that, the same **Merch description**. Any records still missing a state were assigned the value "Foreign". A similar logic was applied to **Merch zip**: it was first filled using the first observed ZIP from the same **Merchnum** or **Merch description**. If the ZIP was still missing but the **Merch state** was known, the ZIP was imputed using the most frequent (mode) ZIP code found within that state.

Variable Creation

To enhance the model's predictive power, the raw, transaction-level data was engineered into a comprehensive set of features. This process is critical as it moves beyond static data points to capture behavioral history, transactional velocity, and contextual anomalies. The new features are grouped into the following categories:

1. Core & Target-Encoded Features

- **Core Fields:** The analysis is built upon fundamental fields such as **Amount** and the **Fraud** label, which serves as the binary target for our supervised learning model.
- **Target-Encoded (TE) Features:** To leverage categorical data, variables like Merch_state were converted into numerical features using target mean encoding. This technique calculates the historical fraud rate for each category (e.g., the average fraud rate for all transactions in a specific state). This provides the model with a powerful, pre-calculated risk signal based on the historical behavior associated with that entity.

2. Historical Aggregations (Time-Window Features)

To establish a baseline of "normal" behavior, a series of rolling aggregations were calculated for key entities (like Cardnum and Merchnum) over various time windows (e.g., 1, 3, 7, 14, 30, and 60 days).

- **Count Features (_count):** These capture transaction frequency (e.g., Cardnum_count_7). The business rationale is that fraudulent activity often occurs in high-frequency bursts, making a sudden spike in count a strong indicator of risk.
- **Amount Aggregation Features (_avg, _med, _max, _total):** These features establish a historical spending profile for an entity.
    - **_avg (Average) and _med (Median)** define the "normal" transaction value, with the median being more robust to outliers.
    - **_max (Maximum)** tracks the highest transaction amount seen, allowing the model to flag new "high-water mark" transactions.
    - **_total (Total)** monitors the cumulative monetary value over a period, which can spike significantly during a fraud event.

3. Contextual Ratio Features

These features are among the most powerful predictors, as they put the current transaction into its historical context. They measure relative change by comparing the current transaction's Amount to the historical aggregations created above.

- **Example:** The feature **Cardnum_actual/avg_30** (the ratio of the current amount to the card's 30-day average) contextualizes the spending. A $500 transaction is normal

for a card that averages $450 but is a powerful anomaly for a card that averages $50.

4. Velocity and Recency Features

This category measures the time elapsed since the last activity for a specific entity.

- **Example:** **Cardnum_day_since** tracks the number of days since the card was last used. The business rationale is that a long period of dormancy followed by a sudden, high-value transaction is a common fraudulent pattern.

5. Behavioral and Anomaly Flags

Finally, a set of custom boolean flags and risk indicators were engineered to capture specific, known-fraud tactics that are not easily captured by the features above.

- **Examples:** This includes features like **state_inconsistency_flag**, which identifies geographically impossible transactions (e.g., a card used in two distant states on the same day), and **Dow_Risk**, which assigns risk based on the day of the week, as fraud patterns can vary by day.

Feature Selection

Methodology

Our feature selection process is a two-stage approach. First, we apply a univariate filter to rank each variable's individual predictive power. We use the **Kolmogorov-Smirnov (KS) statistic**, which is a measure that quantifies **how well a single variable can separate** our "fraud" and "non-fraud" transactions. A higher KS score means the variable is more effective at distinguishing between these two groups. The variables are ranked by their KS scores, and the **top 400** are selected. This threshold is an empirical heuristic that has consistently yielded strong performance in prior analyses.

Second, this set of 400 variables is passed into a **Stepwise Feature Selection (SFS)** process, implemented using the mlxtend library. A fast, simple, non-linear model, specifically a **LightGBM Classifier** (LGBMClassifier(n_estimators=20, num_leaves=4)), is used as the estimator to evaluate feature subsets efficiently.

To provide more flexibility for the final modeling stage and avoid being overly conservative, we ultimately retain a slightly larger set of variables than the SFS algorithm's strict recommendation. This ensures potentially valuable predictors are not prematurely eliminated.

Final Variable List

Fig 3. Final Selected Variables

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.518606
2	Card_Merchdesc_State_total_3	0.259921
3	card_merch_count_0_by_60	0.236612
4	Cardnum_count_0_by_60	0.329398
5	card_state_total_amount_1_by_60	0.348891
6	Card_Merchdesc_Zip_total_30	0.25332
7	card_state_max_14	0.229356
8	Merchnum_desc_total_1	0.243629
9	card_state_total_14	0.264225
10	Card_dow_total_amount_0_by_60	0.32686
11	Card_dow_vdratio_0by14	0.502657
12	Card_dow_count_0_by_14	0.255812
13	card_zip_total_amount_1_by_30	0.281011
14	Cardnum_max_1	0.504805
15	card_merch_total_amount_1_by_60	0.305567
16	Card_dow_unique_count_for_merch_state_14	0.417871
17	Cardnum_max_0	0.492316
18	Cardnum_variability_max_7	0.313458
19	Merchdesc_Zip_total_0	0.233904
20	Cardnum_unique_count_for_card_zip_7	0.474424

Note: the filter score as shown above is the D-statistic from the KS test.

Preliminary Model Explores

Algorithm Descriptions

Logistic Regression (log reg)

Logistic Regression is a foundational statistical model used for binary classification, such as predicting "fraud" vs. "non-fraud." It works by estimating the probability of an outcome based on a set of input variables. It's known for being fast, easy to interpret, and providing a good baseline, though it may not capture highly complex, non-linear patterns as well as tree-based models.

Decision Tree (DT)

A Decision Tree is a simple, flowchart-like model that makes predictions by asking a series of "if-then" questions about the data's features. It splits the data into branches, with each end-point (a "leaf") representing a final decision. While very easy to visualize and understand, a single tree is often prone to "overfitting" (memorizing the training data) and can be unstable.

Random Forest (RF)

A Random Forest is an "ensemble" model that improves upon the single Decision Tree. It works by building a large number of individual Decision Trees during training, each on a different random subset of the data and features. To make a prediction, it takes a "majority vote" from all the trees. This approach significantly increases accuracy and prevents the overfitting seen in single trees.

LightGBM (LGBM)

LightGBM (Light Gradient Boosting Machine) is a high-performance "gradient boosting" model. Boosting works by building models sequentially, where each new model (a small tree) is trained to correct the errors of the previous ones. LGBM is known for being extremely fast, memory-efficient, and highly accurate, especially on large datasets.

LGBM with SMOTE

This isn't a new model, but rather the application of a specific data-handling technique before training the LGBM model. SMOTE (Synthetic Minority Over-sampling Technique) is used to fix class imbalance (e.g., when there are very few fraud cases). It creates new, "synthetic" examples of the minority class (fraud) to give the model more balanced data to learn from.

Neural Network (NN)

A Neural Network is a model inspired by the structure of the human brain, composed of interconnected layers of "neurons" (nodes). Data is fed into the first layer, passes through one or more "hidden" layers where complex patterns are learned, and produces a final prediction. NNs are very powerful and can capture highly complex relationships, but can also be computationally expensive and difficult to interpret.

CatBoost

CatBoost (Categorical Boosting) is another advanced gradient boosting model, similar to LGBM. Its key feature is its exceptional ability to automatically and intelligently handle categorical features (like 'Merch state' or 'ZIP code') without requiring extensive manual pre-processing, which often leads to improved accuracy and simpler model development.

LGBM sampled

This entry describes an LGBM model that was trained on a "sampled" dataset. This is another method to handle class imbalance, but it's more general than SMOTE. It typically means either "undersampling" (randomly removing many non-fraud cases) or "oversampling" (randomly duplicating the few fraud cases) to create a more balanced training set for the model.

XGBoost (XGB)

XGBoost (Extreme Gradient Boosting) is another powerful and widely-used gradient boosting framework, and a direct competitor to LGBM and CatBoost. It builds trees sequentially to correct prior errors and is renowned for its high predictive accuracy and built-in regularization, which helps prevent overfitting. It has been a dominant model in many machine learning competitions.

Comparative Performance Table

In this section, we seek to compare different variations of the models mentioned above.

In selecting the primary evaluation metric for this model, our focus was placed on the **Fraud Detection Rate (FDR)**, which is operationally equivalent to **Recall (True Positive / (True Positive + False Negative))**. This was a strategic business decision driven by the asymmetric costs associated with misclassification in real-time fraud detection. A **False Negative**—where a fraudulent transaction is missed—results in an immediate, direct, and often irreversible financial loss. Conversely, a **False Positive**—where a legitimate transaction is incorrectly declined—creates a poor customer experience and operational costs, but this "customer friction" is considered a more manageable business problem

than the direct financial loss from uncaught fraud. Therefore, our priority is to maximize Recall (FDR) to ensure we are catching the highest possible percentage of actual fraudulent activity.

Each model's score distribution is presented across three distinct datasets:

- The **Training (Trn) set** (blue) is the historical data used to build and train the models.
- The **Testing (Tst) set** (red) is a hold-out set of unseen data from the same time period, used to validate the model's accuracy and generalization.
- The **Out-of-Time (OOT) set** (green) consists of data from a subsequent time period, offering the most critical benchmark of how the model is expected to perform on new, future data.

The model selection process was guided by three key criteria:

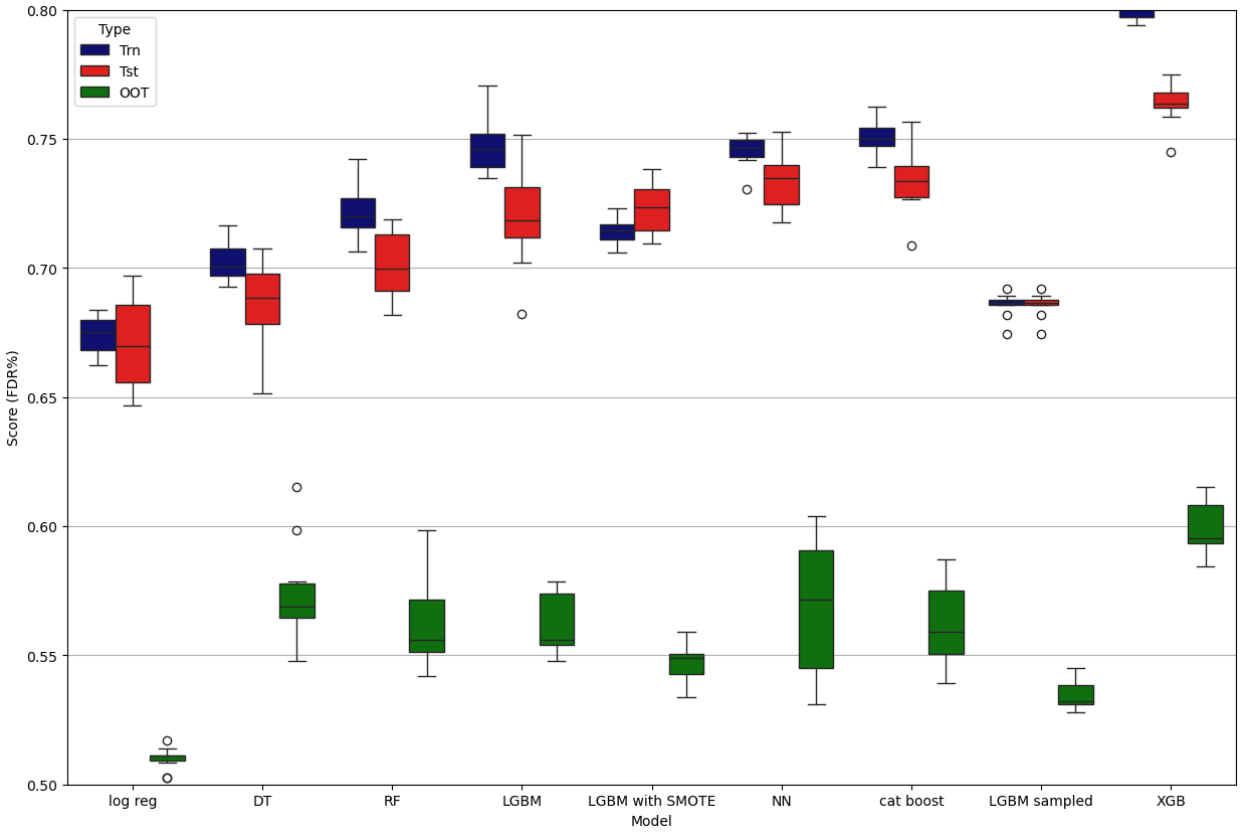
1. **Performance:** Achieving the highest possible FDR% on the **Testing (Tst)** and, most importantly, the **Out-of-Time (OOT)** sets.
2. **Stability:** Ensuring a minimal gap between the **Training (Trn)** and **Testing (Tst)** scores. A large gap would suggest "overfitting," meaning the model has memorized the training data's noise rather than its true patterns.
3. **Model Simplicity:** When performance is comparable, preference is given to the less complex model, as it is generally more computationally efficient, easier to interpret, and more robust to maintain.

Fig 4. Documentation of Models Explored with Different Hyperparameter Mix

Logistic Regression	Number of variables	penalty	C	solver	l1_ratio			Train	Test	OOT
1	15	l2	1	lbfgs	None			0.67	0.679	0.508
2	15	None	1	lbfgs	None			0.669	0.678	0.507
3	15	l1	1	saga	None			0.674	0.67	0.507
4	15	l2	1	saga	None			0.672	0.675	0.506
5	15	elasticnet	1	saga	1			0.67	0.677	0.506
Decision Tree	Number of variables	criterion	splitter	max_depth	min_samples_split	min_samples_leaf		Train	Test	OOT
1	15	gini	best	10	100	50		0.738	0.713	0.536
2	15	gini	best	10	200	100		0.702	0.682	0.574
3	15	gini	best	7	100	50		0.717	0.679	0.558
4	15	gini	best	7	200	100		0.691	0.679	0.556
Random Forest	Number of variables	n_estimators	criterion	max_depth	max_features	min_samples_leaf		Train	Test	OOT
1	15	20	gini	10	8	80		0.719	0.699	0.57
2	15	20	gini	10	sqrt	80		0.715	0.699	0.544
3	15	20	gini	10	log2	80		0.714	0.698	0.548
4	15	100	gini	10	8	80		0.722	0.705	0.565
5	15	100	gini	20	8	80		0.723	0.706	0.565
Boosted Tree	Number of variables	booster	max_depth	learning_rate	n_estimators	subsample		Train	Test	OOT
1	15	gbtree	2	0.1	80	1		0.72	0.705	0.545
2	15	gbtree	2	0.1	80	0.7		0.716	0.715	0.546
3	15	gbtree	2	0.01	80	1		0.668	0.665	0.502
4	15	gbtree	2	0.1	300	1		0.788	0.75	0.578
5	15	gbtree	4	0.1	80	0.7		0.799	0.771	0.598
6	15	gbtree	4	0.1	300	0.7		0.903	0.787	0.614
Neural Network	Number of variables	hidden_layer_sizes	activation	alpha	learning_rate	learning_rate_init	max_iter	Train	Test	OOT
1	15	(10,10)	relu	0.01	constant	0.02	200	0.737	0.724	0.567
2	15	(10,10)	relu	0.01	constant	0.001	200	0.749	0.736	0.565
3	15	(20,20)	relu	0.01	constant	0.02	200	0.744	0.735	0.583

The following boxplot provides a comparative analysis of nine machine learning models, evaluating their performance based on Fraud Detection Rate (FDR%).

Fig 5. Comparison of Performance of Various Models



Final Model Performance

Final Model Architecture and Hyperparameters

The final model for this analysis is a Gradient Boosted Tree, implemented via the **XGBoost** framework. This model was selected for its **superior performance on the testing set** and its strong stability, evidenced by a **minimal performance difference** between the training and testing data. While the performance difference might seem bigger when compared to other models explored, a gap of its size (**0.028**) is generally considered very healthy and **does not suggest significant overfitting**

This is a powerful ensemble technique that builds a series of decision trees sequentially. Unlike a Random Forest which builds independent trees, this model builds one tree, calculates the errors it made, and then builds the next tree specifically to correct those errors. This process is repeated 80 times, with each new tree incrementally improving the model's overall predictive accuracy.

This specific model uses the **15 features** that were selected during the feature selection process. The hyperparameters (non-default choices) were tuned to optimize performance and prevent overfitting, as detailed below:

- **booster = gbtree**: This confirms the model is tree-based. Each "booster" or "weak learner" in the sequential chain is a decision tree.
- **max_depth = 4**: This parameter controls the complexity of each individual tree, limiting it to a maximum of four levels of decision "splits." A depth of 4 was chosen as a balance between capturing complex variable interactions and preventing overfitting. If the trees were too deep, they might "memorize" the training data and fail to generalize to new, unseen transactions.
- **learning_rate = 0.1**: This parameter, also known as "eta," controls the "step size" or contribution of each new tree to the final prediction. A value of 0.1 means each tree's correction is scaled down, forcing the model to learn more slowly and conservatively. This helps prevent overfitting and leads to a more robust and stable final model.
- **n_estimators = 80**: This defines the total number of trees to be built in the sequence. The model will create 80 trees in total, with each one correcting its predecessor. This number was likely selected through tuning as the point where the model's performance on the test data was maximized.
- **subsample = 0.7**: This is a key anti-overfitting control. Before building each of the 80 trees, the model randomly samples and uses only 70% of the training data. This ensures that each tree learns from a slightly different subset of the data, making the overall model more robust and less likely to be influenced by noise or specific outliers in the training set.

Results: Training Set, Testing Set, Out-of-Time (OOT) Set

The following tables provides a binned performance analysis of the model, ranking all transactions by their predicted fraud score and then grouping them into 100 equal-sized "bins." The analysis is performed on all three data partitions — training, testing, and out-of-time set. The cumulative statistics measure the model's effectiveness as we capture progressively larger percentages of the highest-risk population.

Metric Definitions

- Cumulative % Goods: This column represents the cumulative percentage of all non-fraudulent ("good") transactions captured, proceeding from the highest-risk bin to the lowest.
- Cumulative % Bads (FDR): This is the Fraud Detection Rate (FDR), equivalent to Recall. It measures the cumulative percentage of all fraudulent ("bad") transactions captured by the model at or above a given risk bin. For example, the top 10 bins (10% of the population) capture 93.5% of all fraud in training set.
- KS (Kolmogorov-Smirnov): This statistic identifies the point of maximum separation between the fraudulent and non-fraudulent populations. It is calculated as the difference between the Cumulative % Bads and the Cumulative % Goods at each bin.
- FPR (False Positive Rate): This metric represents the False Positive Rate, calculated here as the ratio of cumulative "goods" to cumulative "bads" captured. It quantifies the number of non-fraudulent transactions flagged for every fraudulent one. For instance, at Bin 10, the FPR of 3.22 indicates that 3.22 legitimate transactions are flagged for every 1 fraudulent transaction caught.

Formulas

- Cumulative % Goods = Cumulative Goods / Total # Goods
- Cumulative % Bads (FDR) = Cumulative Bads / Total # Bads
- KS = Cumulative Bads % - Cumulative Goods %
- FPR = Cumulative Goods / Cumulative Bads

Fig 6a. Final Model Performance on Training Set

Training	# Records	# Goods	# Bads	Fraud Rate
	59,780	58,252	1,528	2.56%

	Bin Statistics								Cumulative Statistics						
Population Bin %	# Records	# Goods	# Bads	% Goods	% Bads	Total # Records	Cumulative Goods	Cumulative Bads	% Goods	% Bads (FDR)	KS	FPR	Fraud Savings	FP Loss	Overall Savings
1	598	39	559	6.521739	93.47826	598	39	559	0.06695	36.58377	36.51682	0.069767	223600	780	222820
2	598	126	472	21.07023	78.92977	1196	165	1031	0.283252	67.47382	67.19057	0.160039	412400	3300	409100
3	597	406	191	68.0067	31.9933	1793	571	1222	0.980224	79.97382	78.9936	0.467267	488800	11420	477380
4	598	528	70	88.29431	11.70569	2391	1099	1292	1.886631	84.55497	82.66834	0.850619	516800	21980	494820
5	598	550	48	91.97324	8.026756	2989	1649	1340	2.830804	87.69634	84.86553	1.230597	536000	32980	503020
6	598	569	29	95.1505	4.849498	3587	2218	1369	3.807595	89.59424	85.78665	1.620161	547600	44360	503240
7	598	585	13	97.82609	2.173913	4185	2803	1382	4.811852	90.44503	85.63317	2.02822	552800	56060	496740
8	597	585	12	97.98995	2.01005	4782	3388	1394	5.816109	91.23037	85.41426	2.430416	557600	67760	489840
9	598	587	11	98.16054	1.839465	5380	3975	1405	6.8238	91.95026	85.12646	2.829181	562000	79500	482500
10	598	588	10	98.32776	1.672241	5978	4563	1415	7.833207	92.60471	84.7715	3.224735	566000	91260	474740
11	598	588	10	98.32776	1.672241	6576	5151	1425	8.842615	93.25916	84.41655	3.614737	570000	103020	466980
12	598	590	8	98.66221	1.337793	7174	5741	1433	9.855456	93.78272	83.92727	4.006281	573200	114820	458380
13	597	593	4	99.32998	0.670017	7771	6334	1437	10.87345	94.0445	83.17106	4.407794	574800	126680	448120
14	598	591	7	98.82943	1.170569	8369	6925	1444	11.888	94.50262	82.61461	4.795706	577600	138500	439100
15	598	595	3	99.49833	0.501672	8967	7520	1447	12.90943	94.69895	81.78952	5.196959	578800	150400	428400
16	598	594	4	99.3311	0.668896	9565	8114	1451	13.92914	94.96073	81.0316	5.592006	580400	162280	418120
17	598	595	3	99.49833	0.501672	10163	8709	1454	14.95056	95.15707	80.20651	5.989684	581600	174180	407420
18	597	593	4	99.32998	0.670017	10760	9302	1458	15.96855	95.41885	79.4503	6.379973	583200	186040	397160
19	598	594	4	99.3311	0.668896	11358	9896	1462	16.98826	95.68063	78.69237	6.76881	584800	197920	386880
20	598	591	7	98.82943	1.170569	11956	10487	1469	18.00282	96.13874	78.13593	7.13887	587600	209740	377860

Fig 6b. Final Model Performance on Testing Set

Testing	# Records	# Goods	# Bads	Fraud Rate
	25,620	25,012	608	2.37%

	Bin Statistics								Cumulative Statistics						
Population Bin %	# Records	# Goods	# Bads	% Goods	% Bads	Total # Records	Cumulative Goods	Cumulative Bads	% Goods	% Bads (FDR)	KS	FPR	Fraud Savings	FP Loss	Overall Savings
1	256	31	225	12.10938	87.89063	256	31	225	0.123941	37.00658	36.88264	0.137778	90000	620	89380
2	256	74	182	28.90625	71.09375	512	105	407	0.419798	66.94079	66.52099	0.257985	162800	2100	160700
3	257	196	61	76.26459	23.73541	769	301	468	1.203422	76.97368	75.77026	0.643162	187200	6020	181180
4	256	220	36	85.9375	14.0625	1025	521	504	2.083	82.89474	80.81174	1.03373	201600	10420	191180
5	256	235	21	91.79688	8.203125	1281	756	525	3.022549	86.34868	83.32614	1.44	210000	15120	194880
6	256	248	8	96.875	3.125	1537	1004	533	4.014073	87.66447	83.6504	1.883677	213200	20080	193120
7	256	247	9	96.48438	3.515625	1793	1251	542	5.001599	89.14474	84.14314	2.308118	216800	25020	191780
8	257	250	7	97.27626	2.723735	2050	1501	549	6.001119	90.29605	84.29493	2.734062	219600	30020	189580
9	256	253	3	98.82813	1.171875	2306	1754	552	7.012634	90.78947	83.77684	3.177536	220800	35080	185720
10	256	252	4	98.4375	1.5625	2562	2006	556	8.02015	91.44737	83.42722	3.607914	222400	40120	182280
11	256	254	2	99.21875	0.78125	2818	2260	558	9.035663	91.77632	82.74065	4.050179	223200	45200	178000
12	256	254	2	99.21875	0.78125	3074	2514	560	10.05118	92.10526	82.05409	4.489286	224000	50280	173720
13	257	253	4	98.44358	1.55642	3331	2767	564	11.06269	92.76316	81.70047	4.906028	225600	55340	170260
14	256	254	2	99.21875	0.78125	3587	3021	566	12.0782	93.09211	81.0139	5.337456	226400	60420	165980
15	256	254	2	99.21875	0.78125	3843	3275	568	13.09372	93.42105	80.32734	5.765845	227200	65500	161700
16	256	256	0	100	0	4099	3531	568	14.11722	93.42105	79.30383	6.216549	227200	70620	156580
17	256	255	1	99.60938	0.390625	4355	3786	569	15.13673	93.58553	78.44879	6.653779	227600	75720	151880
18	257	255	2	99.22179	0.77821	4612	4041	571	16.15625	93.91447	77.75823	7.077058	228400	80820	147580
19	256	253	3	98.82813	1.171875	4868	4294	574	17.16776	94.40789	77.24014	7.480836	229600	85880	143720
20	256	254	2	99.21875	0.78125	5124	4548	576	18.18327	94.73684	76.55357	7.895833	230400	90960	139440

Fig 6c. Final Model Performance on Out-of-Time Validation Set

OOT	# Records	# Goods	# Bads	Fraud Rate
	12,637	12,281	356	2.82%

	Bin Statistics					Cumulative Statistics						
Population Bin %	# Records	# Goods	# Bads	% Goods	% Bads	Total # Records	Cumulative Goods	Cumulative Bads	% Goods	% Bads (FDR)	KS	FPR
1	126	18	108	14.28571	85.71429	126	18	108	0.146568	30.33708	30.19051	0.166667
2	127	53	74	41.73228	58.26772	253	71	182	0.578129	51.1236	50.54547	0.39011
3	126	85	41	67.46032	32.53968	379	156	223	1.270255	62.64045	61.37019	0.699552
4	126	107	19	84.92063	15.07937	505	263	242	2.141519	67.97753	65.83601	1.086777
5	127	109	18	85.82677	14.17323	632	372	260	3.029069	73.03371	70.00464	1.430769
6	126	111	15	88.09524	11.90476	758	483	275	3.932904	77.24719	73.31429	1.756364
7	127	118	9	92.91339	7.086614	885	601	284	4.893738	79.77528	74.88154	2.116197
8	126	121	5	96.03175	3.968254	1011	722	289	5.879	81.17978	75.30078	2.49827
9	126	123	3	97.61905	2.380952	1137	845	292	6.880547	82.02247	75.14192	2.893836
10	127	121	6	95.27559	4.724409	1264	966	298	7.865809	83.70787	75.84206	3.241611
11	126	125	1	99.20635	0.793651	1390	1091	299	8.883641	83.98876	75.10512	3.648829
12	126	123	3	97.61905	2.380952	1516	1214	302	9.885189	84.83146	74.94627	4.019868
13	127	120	7	94.48819	5.511811	1643	1334	309	10.86231	86.79775	75.93545	4.317152
14	126	125	1	99.20635	0.793651	1769	1459	310	11.88014	87.07865	75.19851	4.706452
15	127	125	2	98.4252	1.574803	1896	1584	312	12.89797	87.64045	74.74248	5.076923
16	126	125	1	99.20635	0.793651	2022	1709	313	13.9158	87.92135	74.00554	5.460064
17	126	126	0	100	0	2148	1835	313	14.94178	87.92135	72.97957	5.86262
18	127	127	0	100	0	2275	1962	313	15.9759	87.92135	71.94545	6.268371
19	126	126	0	100	0	2401	2088	313	17.00187	87.92135	70.91948	6.670927
20	126	121	5	96.03175	3.968254	2527	2209	318	17.98713	89.32584	71.33871	6.946541

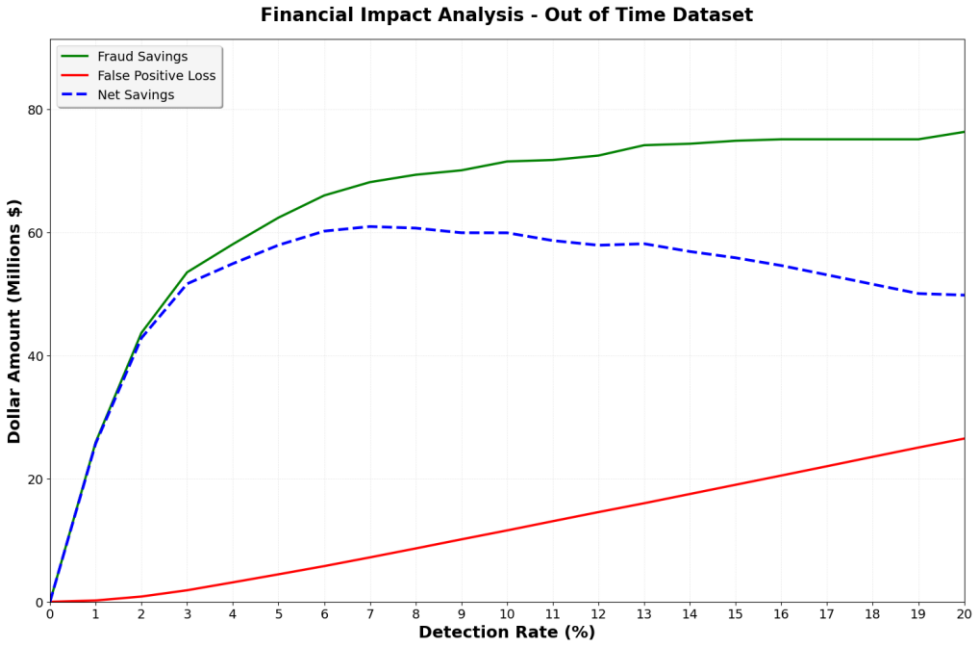
Financial Curves and Recommended Cutoff

Financial Impact

The following plot illustrates the financial trade-off of the model's implementation on the Out-of-Time (OOT) dataset. The x-axis represents the percentage of all transactions flagged (Detection Rate), while the y-axis shows the corresponding dollar impact.

- The **Fraud Savings** (green line) shows the cumulative dollar amount of fraud prevented. It rises steeply at low detection rates, indicating the model successfully identifies high-value fraud first, and then begins to flatten out as most of the fraud is captured.
- The **False Positive Loss** (red line) represents the cumulative cost of reviewing legitimate transactions, which increases steadily as we flag more transactions.
- The **Net Savings** (blue dashed line) is the key business metric, calculated as Fraud Savings minus False Positive Loss. This curve peaks at a **detection rate of approximately 7%**, achieving a maximum net savings of just over $60 million. Beyond this optimal point, the costs incurred from reviewing more false positives begin to outweigh the savings from catching the small amount of remaining fraud, causing the total net savings to decline.

Fig 7. Final Model Performance on Out-of-Time Validation Set



Recommended Cutoff Strategy

The recommended strategy is to implement a **6% detection rate**. Although the theoretical maximum net savings occurs at approximately 7%, analysis indicates **strong diminishing returns** beyond the 6% mark. Adopting a more conservative 6% threshold is a strategic decision to mitigate the negative business impact of excessive false positives, such as customer friction and potential brand risk. At this recommended rate, the fraud detection model is projected to yield **$60.2 million in net savings**.

Summary

This report detailed the end-to-end development of a real-time fraud detection model, beginning with data cleaning and exploration of 98,393 transaction records. The process involved a multi-step imputation strategy for missing merchant location data and the creation of rule-based flags for known anomalies, such as recurring FedEx charges. The core of the project was a comprehensive feature engineering process that transformed raw data into powerful predictors. This included creating target-encoded features (e.g., Merch_state_TE), historical aggregations over rolling time windows (e.g., Cardnum_count_7), and contextual ratio features (e.g., Cardnum_actual/avg_30) to capture behavioral and velocity-based patterns.

A two-stage feature selection process was employed, starting with a Kolmogorov-Smirnov (KS) univariate filter to identify the top 400 variables, which were then refined using Stepwise Feature Selection (SFS) to produce a final set of 15 predictors. Nine different machine learning models were explored, with a focus on Fraud Detection Rate (FDR/Recall) as the primary business metric. An XGBoost (gbtree) model was selected as the champion for its superior and stable performance. The final model was tuned with specific hyperparameters (max_depth=4, n_estimators=80, subsample=0.7) to optimize performance while preventing overfitting.

The final model's performance on the critical Out-of-Time (OOT) dataset is highly effective, achieving an **FDR of 62.64% while reviewing only the top 3%** of the highest-risk transactions. A full financial impact analysis was conducted, leading to a recommended business strategy of implementing a **6% detection rate**. This cutoff is projected to yield **$60.2 million in net savings**, representing the optimal balance between maximizing fraud capture and minimizing the operational costs of reviewing false positives.

For future work, this model's performance should be monitored for drift and periodically retrained as new data and fraud patterns emerge. Further gains could potentially be achieved by exploring more advanced hyperparameter tuning, testing new feature interactions, or developing separate, specialized models for different merchant segments or transaction types.

Appendix

Full Data Quality Report (DQR)

1. Data Description

The dataset is **Card Transactions Data**, which includes merchant identifiers, transaction types, and a binary fraud label. The data came from around 100,000 real U.S. applications **over 8 months**. There are **10 fields** and **98,393 records**.

2. Summary Tables

Appendix I. Numeric Fields Table

Field Name	Field Type	# Records Have Values	% Populated	# Zeros	Min	Max	Mean	Standard Deviation	Most Common
Amount	numeric	98393	100.00%	0	0.01	3102046	424.2909	9922.44	3.62

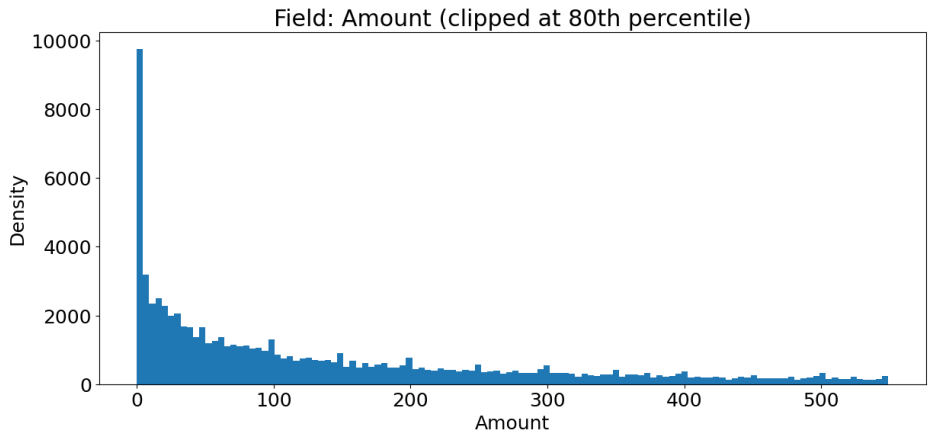
Appendix II. Categorical Fields Table

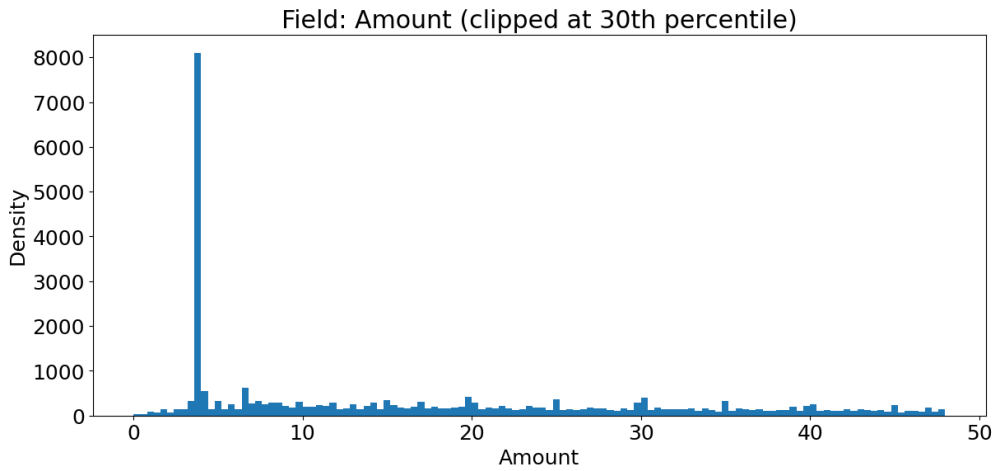
Field Name	Field Type	# Records Have Values	% Populated	# Zeros	# Unique Values	Most Common
Date	Categorical	98393	100.00%	0	365	2/28/10
Merchnum	Categorical	94970	96.52%	0	13091	930090121224
Merch description	Categorical	98393	100.00%	0	13126	GSA-FSS-ADV
Merch state	Categorical	97181	98.77%	0	227	TN
Transtype	Categorical	98393	100.00%	0	4	P
Recnum	Categorical	98393	100.00%	0	98393	1
Fraud	Categorical	98393	100.00%	95901	2	0
Merch zip	Categorical	93664	95.19%	0	4567	38118
Cardnum	Categorical	98393	100.00%	0	1645	5142148452

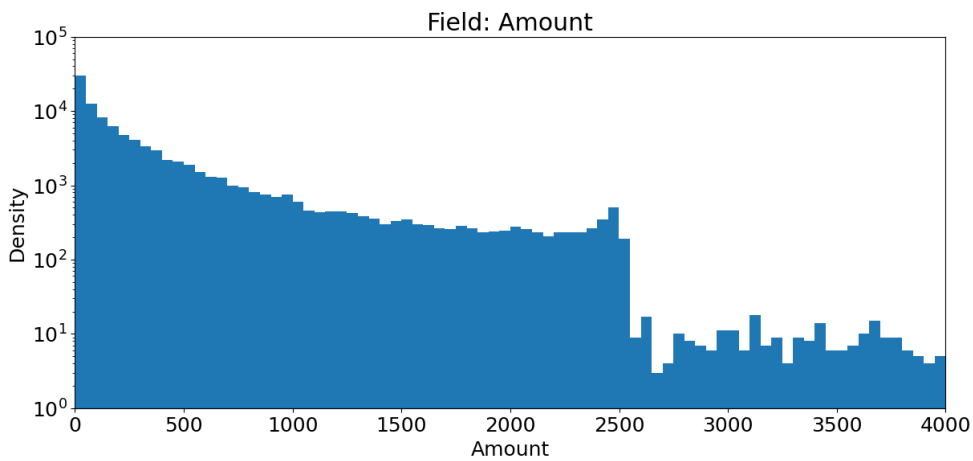
3. Visualization of Each Field

1) Field Name: Amount

Description: Amount of each transaction. Strictly positive. Many transaction amounts are between $3-$4. When logged, the distribution shows a drop off in number of transactions for amount bigger then ~$2500.

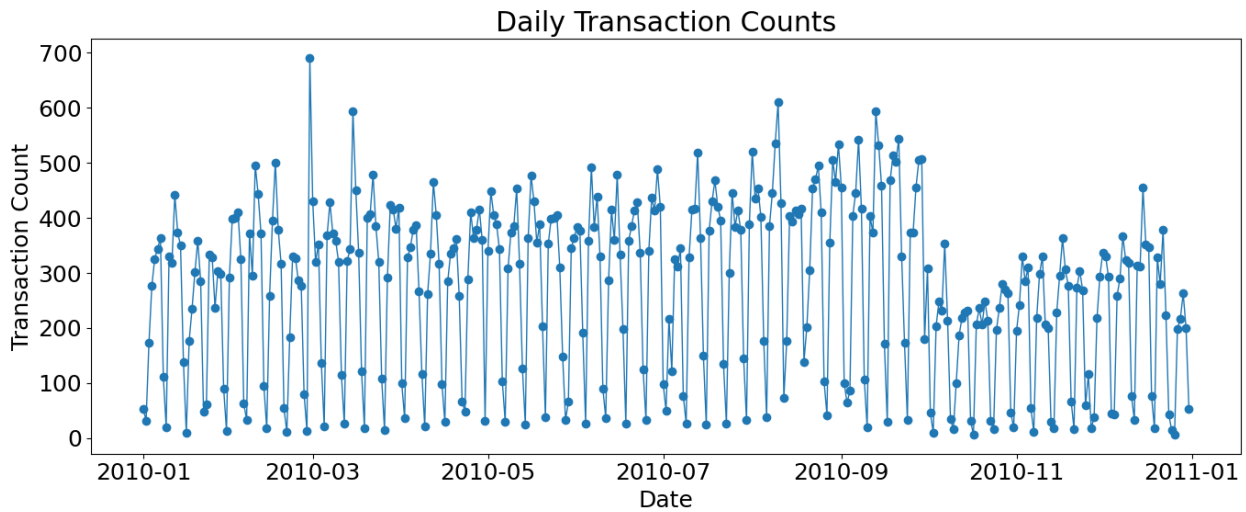


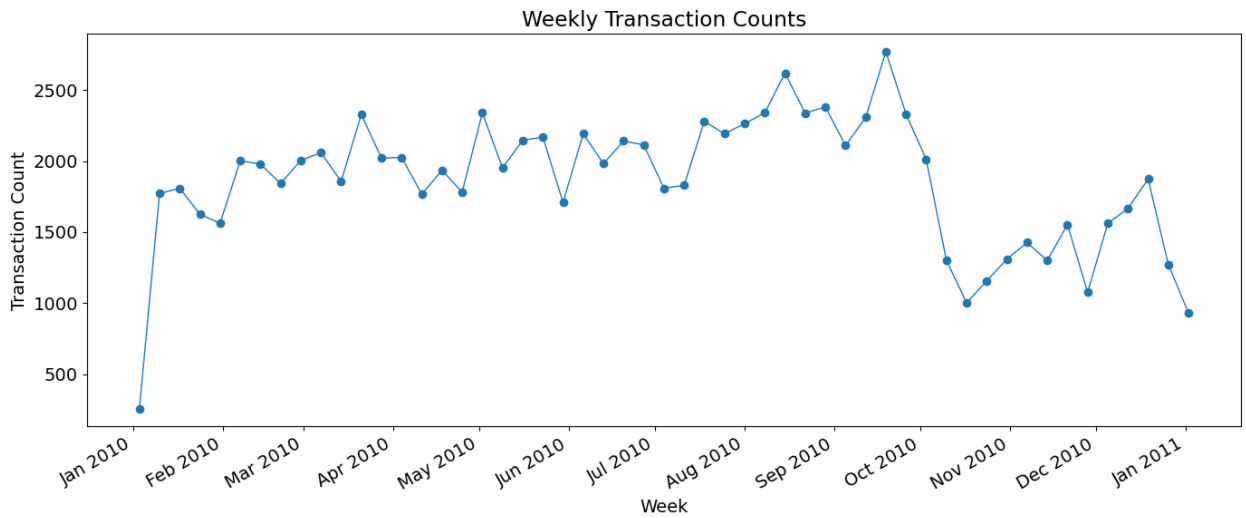




2) **Field Name: date**

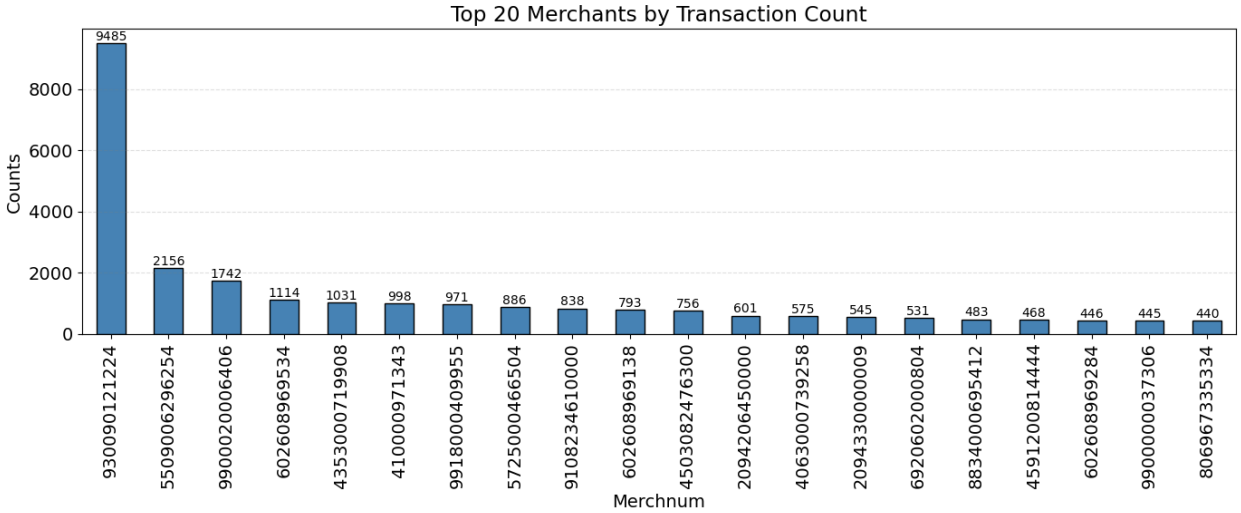
Description: Transaction date. The first distribution shows the number of daily transactions across time. The second distribution shows the number of weekly transactions across time.





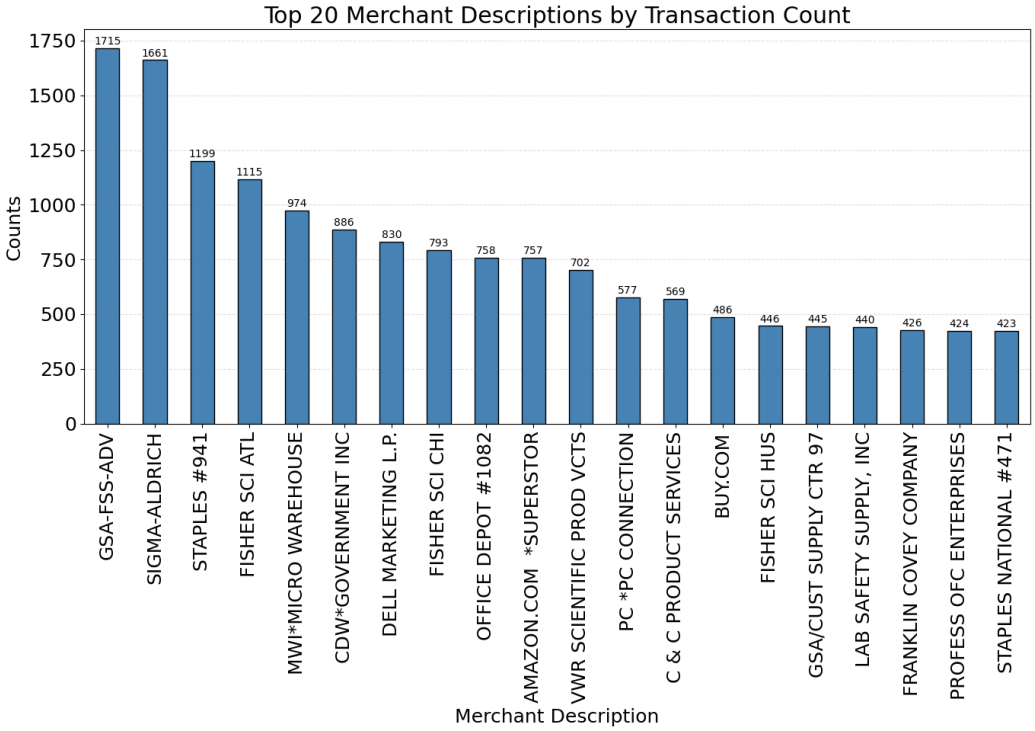
3) **Field Name: Merchnum**

Description: Merchant identification number. The distribution shows the top 20 field values of merchant identification number. The most common ssn is 930090121224, with a total count of 9,485.



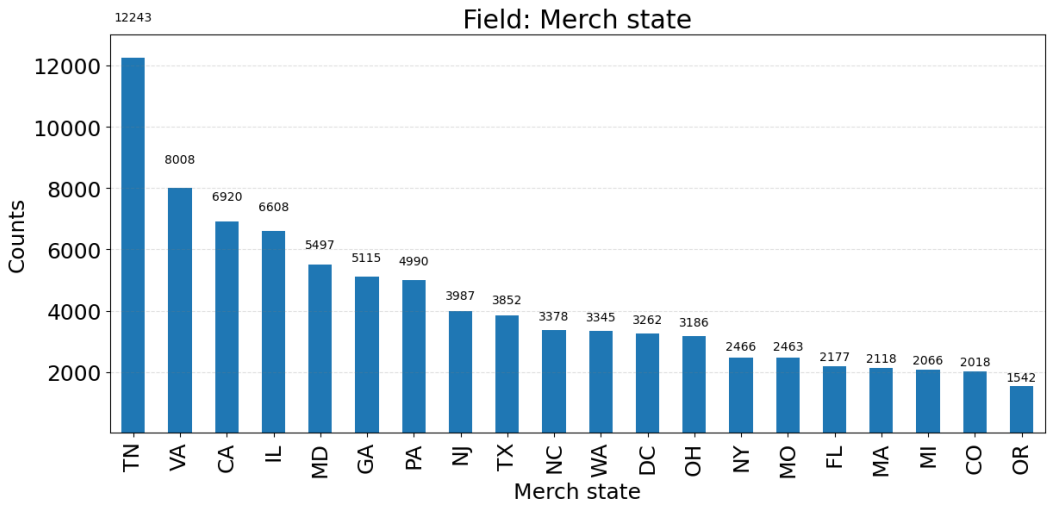
4) **Field Name: Merchant Descriptions**

Description: Description of the merchant. The distribution shows the top 20 field values of merchant description.



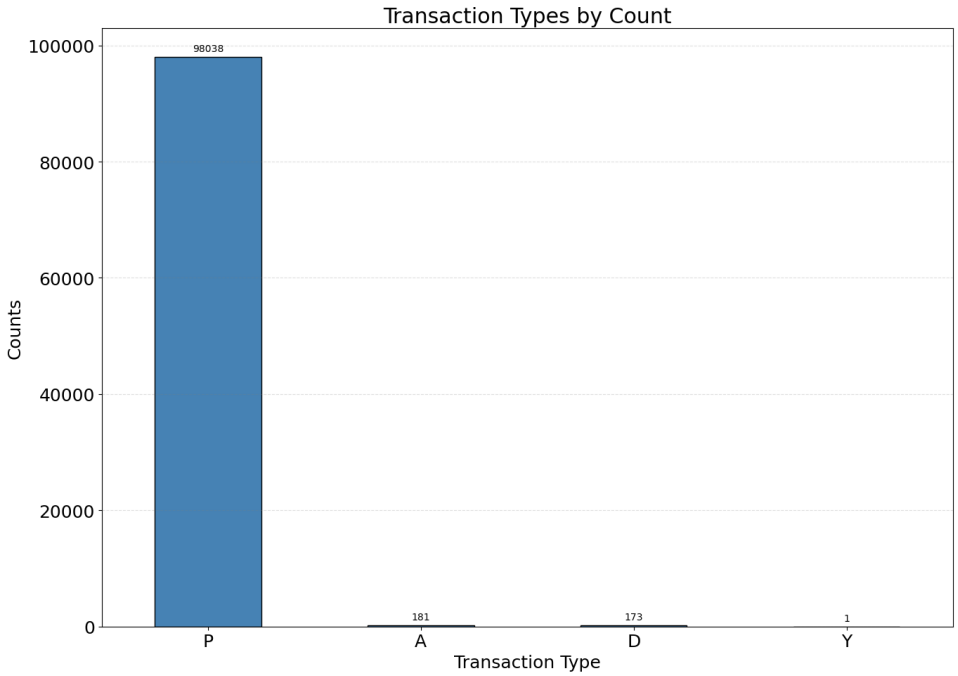
5) **Field Name: Merch state**

Description: The state that the merchant is in. The distribution shows the top 20 field values of the state that the merchant is in. The most common state is TN.



6) **Field Name: Transtype**

Description: Transaction Type. Majority of them is purchase and there are not enough data for other types. Consider removing A, D, and Y.

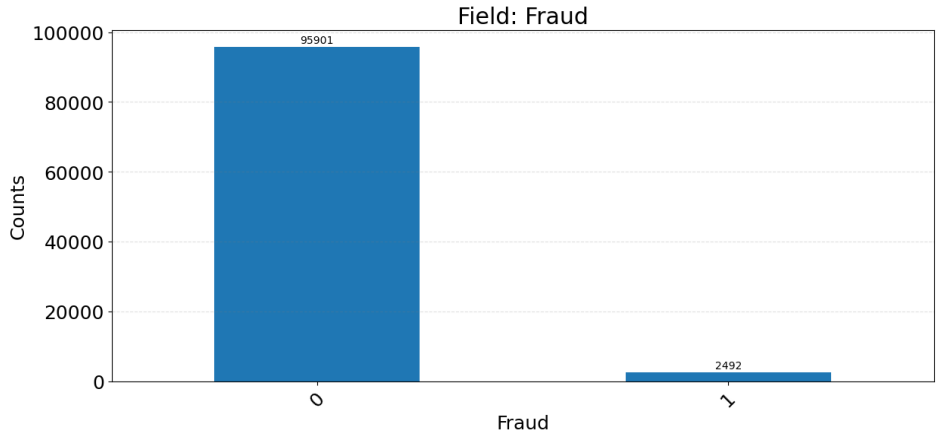


7) **Field Name: Recnum**

   Description: Recnum is the primary key of the table. Unique value for each row. No nulls.

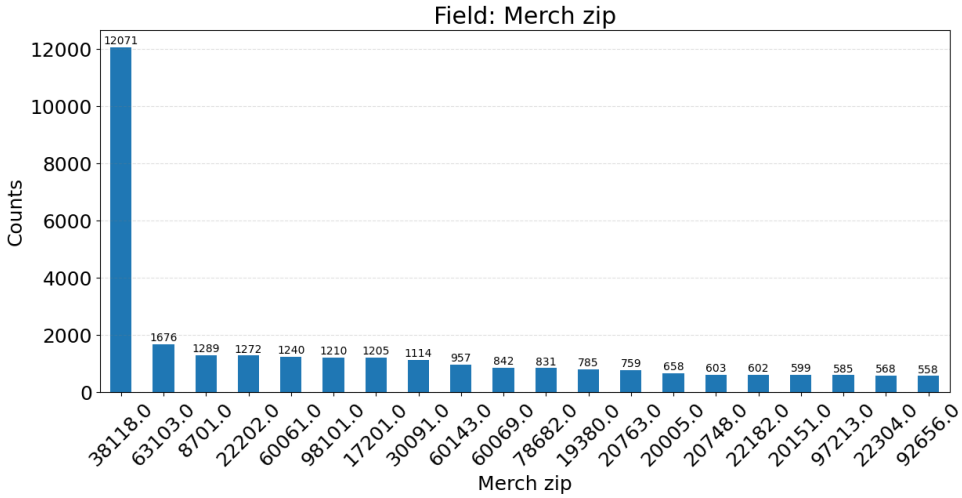
8) **Field Name: Fraud**

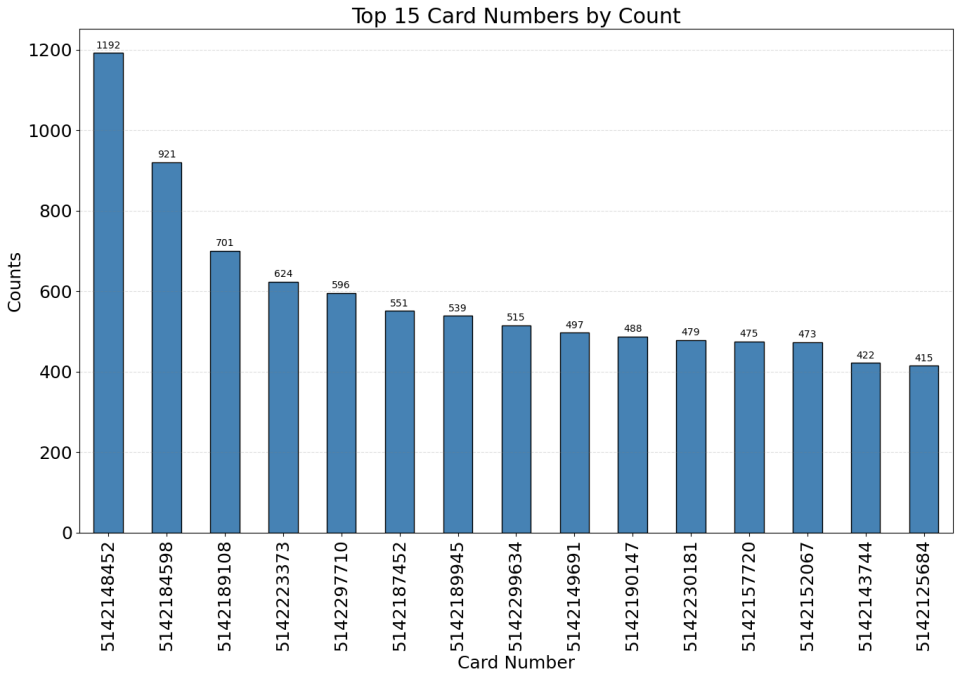
   Description: Label for fraudulent transaction. 1 for affirmative, 0 otherwise. No nulls.



9) **Field Name: Merchzip**

   Description: Merchant Zipcode. *I should have changed it to a string here. Append 0 in the end if needed.*



10) Field Name: Cardnum

Description: This person with the card number 5142148452 has conducted a lot of transactions.